

Lexical Analyzer & Token Counter

Objective

To develop a C program that reads a source-code file and performs lexical analysis by identifying, classifying, and counting different types of tokens such as keywords, identifiers, operators, constants/literals, and separators/delimiters.

Problem Statement

Develop a program that reads a source-code file as input and analyzes its contents. The program should identify different lexical tokens and display the token, its type, and the total count of each token category.

The tokens to be identified are:

- Keywords
- Identifiers
- Operators
- Constants/Literals
- Separators/Delimiters

Algorithm

1. Start the program.
2. Ask the user to enter the name of the source-code file.
3. Open the file in read mode.
4. Read the file character by character.
5. Ignore whitespace characters.
6. If a character is a letter or underscore, collect subsequent letters, digits, or underscores to form a word.
7. Check whether the word is a C keyword.
8. If it is a keyword, classify it as a **Keyword**; otherwise classify it as an **Identifier**.
9. If a character is a digit, collect the complete numeric value and classify it as a **Constant/Literal**.
10. Check for single-character and multi-character operators such as +, -, *, /, =, ==, <=, >=, etc.
11. Check for separators/delimiters such as (,), {, }, [,], ;, and ,.
12. Display every identified token along with its token type.
13. Maintain counters for each token category.
14. Display the total number of tokens in each category.
15. Close the file.
16. Stop the program.

Source Code

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

#define MAX 100

char *keywords[] = {

    "int", "float", "char", "double", "if", "else",

    "for", "while", "do", "return", "void", "break",

    "continue", "switch", "case", "default"

};

int keywordCount = 16;

int isKeyword(char *word)

{    int i;

    for (i = 0; i < keywordCount; i++)

    {

        if (strcmp(word, keywords[i]) == 0)

            return 1;

    }

    return 0;

}

int main()

{

    FILE *fp;

    char filename[100];

    char ch, next;

    char word[100];

    int i;
```

```
int keywords_count = 0;

int identifiers = 0;

int operators = 0;

int constants = 0;

int string_literals = 0;

int separators = 0;

int comments = 0;

int special_symbols = 0;

printf("Enter input file name: ");

scanf("%s", filename);

fp = fopen(filename, "r");

if (fp == NULL)

{

    printf("File cannot be opened.\n");

    return 1;

}

printf("\nTOKEN TYPE\n");

printf("-----\n");

while ((ch = fgetc(fp)) != EOF)

{

    if (isspace(ch))

        continue;

    if (isalpha(ch) || ch == '_')

    {

        i = 0;

        while (isalnum(ch) || ch == '_')
```

```
{  
    word[i++] = ch;  
    ch = fgetc(fp);  
}  
word[i] = '\0';  
if (isKeyword(word))  
{  
    printf("%-20s Keyword\n", word);  
    keywords_count++;  
}  
else  
{  
    printf("%-20s Identifier\n", word);  
    identifiers++;  
}  
ungetc(ch, fp);  
}  
else if (isdigit(ch))  
{  
    i = 0;  
    while (isdigit(ch) || ch == '.')  
    {  
        word[i++] = ch;  
        ch = fgetc(fp);  
    }  
word[i] = '\0';
```

```
printf("%-20s Constant\n", word);

    constants++;

ungetc(ch, fp);

}

else if (ch == "")

{

    i = 0;

    word[i++] = ch;

    ch = fgetc(fp);

    while (ch != "" && ch != EOF)

    {

        word[i++] = ch;

        ch = fgetc(fp);

    }

    word[i++] = "";

    word[i] = '\0';

    printf("%-20s String Literal\n", word);

    string_literals++;

}

else if (ch == '/')

{

    next = fgetc(fp);

    if (next == '/')

    {

        comments++;

    }

}

while ((ch = fgetc(fp)) != '\n' && ch != EOF)
```

```
printf("%-20s Comment\n", "/");  
  
    }  
else if (next == '*')  
    {  
        comments++;  
        while (1)  
        {  
            ch = fgetc(fp);  
            if (ch == '*' && (next = fgetc(fp)) == '/')  
                break;  
            if (ch == EOF)  
                break;  
        }  
        printf("%-20s Comment\n", "/* */");  
    }  
else  
    {  
        printf("%-20s Operator\n", "/");  
        operators++;  
        ungetc(next, fp);  
    }  
}  
else if (ch == '+' || ch == '-' || ch == '*' ||  
        ch == '=' || ch == '<' || ch == '>' ||  
        ch == '%' || ch == '!' || ch == '&' ||  
        ch == '|')
```

```
{
    next = fgetc(fp);
    if ((ch == '+' && next == '+') ||
        (ch == '-' && next == '-') ||
        (ch == '=' && next == '=') ||
        (ch == '!' && next == '=') ||
        (ch == '<' && next == '=') ||
        (ch == '>' && next == '=') ||
        (ch == '&' && next == '&') ||
        (ch == '|' && next == '|'))
    {
        printf("%c%c Operator\n", ch, next);
        operators++;
    }
    else
    {
        printf("%-20c Operator\n", ch);
        operators++;
        ungetc(next, fp);
    }
}

else if (ch == '(' || ch == ')') ||
    ch == '{' || ch == '}' ||
    ch == '[' || ch == ']' ||
    ch == ';' || ch == ',' ||
    ch == ':')
```

```
{  
    printf("%-20c Separator\n", ch);  
    separators++;  
}  
else  
{  
    printf("%-20c Special Symbol\n", ch);  
    special_symbols++;  
}  
}  
printf("-----\n");  
printf("\nToken Count\n");  
printf("Keywords    : %d\n", keywords_count);  
printf("Identifiers   : %d\n", identifiers);  
printf("Operators    : %d\n", operators);  
printf("Constants    : %d\n", constants);  
printf("String Literals: %d\n", string_literals);  
printf("Separators    : %d\n", separators);  
printf("Comments     : %d\n", comments);  
printf("Special Symbols: %d\n", special_symbols);  
fclose(fp);  
return 0;  
}
```

Sample Input

Create a file named **sample.c** containing:

```
int sum = a + b;
```



```
float average = sum / 2.0;
```

```
// Calculate average
```

```
if (average > 50)
```

```
    printf("Pass");
```

When executing the program:

Enter source code file name: sample.c

Output

```
Enter source file name: sample.c

TOKEN TYPE
-----
int           Keyword
sum           Identifier
=             Operator
a             Identifier
+             Operator
b             Identifier
;             Separator
float         Keyword
average       Identifier
=             Operator
sum           Identifier
/             Operator
2.0           Constant
;             Separator
//            Comment
if            Keyword
(             Separator
average       Identifier
>             Operator
50            Constant
)             Separator
printf        Identifier
(             Separator
"Pass"        String Literal
)             Separator
;             Separator
-----

Token Count
Keywords      : 3
Identifiers   : 7
Operators     : 5
Constants     : 2
String Literals: 1
Separators    : 7
```

Token Classification

Token	Token Type
int	Keyword
s	Identifier
=	Operator
(	Separator/Delimiter
a	Identifier
+	Operator
b	Identifier
)	Separator/Delimiter
/	Operator
2	Constant/Literal
;	Separator/Delimiter

Token counts:

• Keywords: 1
• Identifiers: 3
• Operators: 3
• Constants/Literals: 1
• Separators/Delimiters: 3
• Total tokens: 11

Test Cases

Test Case	Input	Expected Result
1	int s = (a + b) / 2;	Keywords, identifiers, operators, constant, and separators are identified
2	int a = 10;	int → Keyword, a → Identifier, 10 → Constant
3	x = y + 5;	x, y → Identifiers, =, + → Operators, 5 → Constant
4	float avg = 25.5;	float → Keyword, avg → Identifier, 25.5 → Constant
5	if (a > b) return a;	if, return → Keywords, a, b → Identifiers, > → Operator

Conclusion

The Lexical Analyzer & Token Counter successfully reads a source-code file, identifies different types of lexical tokens, classifies them, and calculates their frequencies. This program demonstrates the basic working principle of lexical analysis, which is the first phase

of a compiler. It helps in understanding how source code is divided into meaningful tokens for further processing by the compiler.

GitHub Repository:

The screenshot shows the GitHub repository page for 'lexical-analyzer' by user 'Petrin-Prashanthi'. The repository is public and has 0 stars, 0 forks, and 0 watchers. The main branch is 'main' with 1 branch and 0 tags. The repository contains 8 commits. The commit history table is as follows:

Commit	Message	Time
Petrim-Prashanthi	Rename execution.png to screenshots/execution.png	c180f53 · 2 minutes ago
screenshots	Rename execution.png to screenshots/execution.png	2 minutes ago
README.md	Revise README for Lexical Analyzer project	14 minutes ago
lexical.c	Implement lexical analyzer in lexical.c	12 minutes ago
output.txt	Add output.txt with token type analysis	7 minutes ago
sample.c	Add average calculation and pass condition	9 minutes ago

The README section is visible at the bottom of the commit history.

About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
0 forks

Releases
No releases published
[Create a new release](#)

Packages